

Elastic London Meetup

Mon Sept 15, 2025 | 6:30PM

Carly Richmond



What's New?

Elastic 8.19/9.1 🎉

- GA of Cross Cluster Search with ES|QL, Enterprise
- LOOKUP JOINS with ES|QL
- LogsDB & TSDS indexing mode optimisations
- BBQ enabled by default
- ACORN filtered vector search

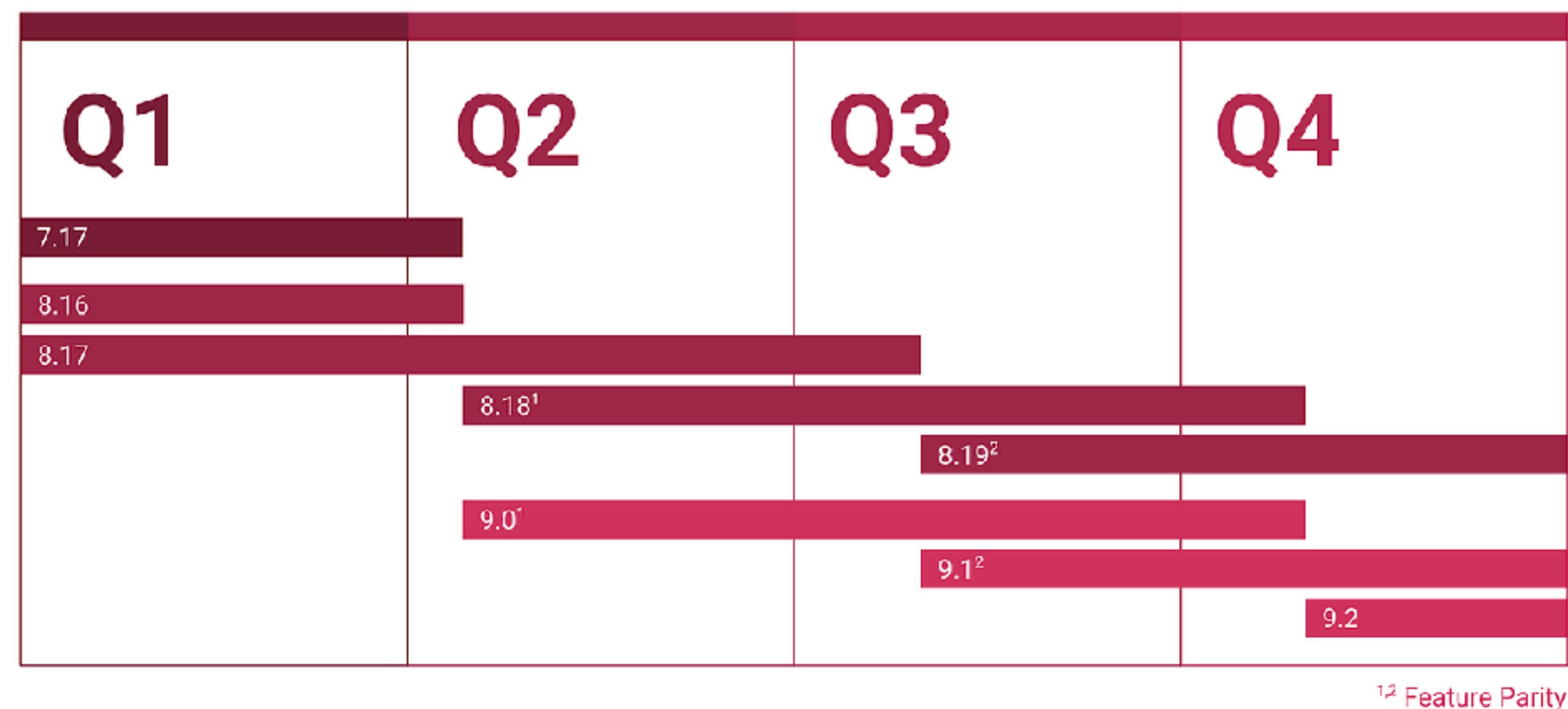
```
FROM kibana_sample_data_logs | WHERE response.keyword != "200"  
| LOOKUP JOIN teams_lkp ON host  
| STATS num = COUNT(*) by host, response.keyword, team  
| SORT num DESC
```

<https://discuss.elastic.co/t/what-s-new-in-elastic-8-19-9-1>

Elastic Stack Releases & Maintenance

Elastic 8.19/9.1 🎉

- Approximately quarterly releases, with 2 minor versions maintained in parallel, to give you more time for upgrades and provide an overlap in maintained versions.
- 8.19 will be maintained for quite a bit longer, but from 9.2 on, all new features will only make it to the 9.x series.
- If you are still on 7.x or earlier, now is a great time to plan moving to a maintained version.



<https://discuss.elastic.co/t/what-s-new-in-elastic-8-19-9-1>

Doing something cool with elastic?

Speak at our meetups!



<https://sessionize.com/elastic-meetups/>

Doing something cool with elastic?

ELASTIC{ON} is coming to London!

26 February 2026

Convene Sancroft, St. Paul's

Register:

<https://www.elastic.co/events/elasticon/london>



<https://sessionize.com/elasticon/>



**Carly
Richmond**
Developer Advocate
Lead,  elastic



**Alam
Ahmed**
Cloud & DevOps
Engineer

WEB UI INSTRUMENTATION WITH

 **OpenTelemetry** &  **elastic**

CARLY RICHMOND

ABOUT ME

- Developer Advocate Lead @
 elastic
- Frontend Engineer, Speaker & Blogger



SCAN ME



USER

FIRST PORT OF CALL



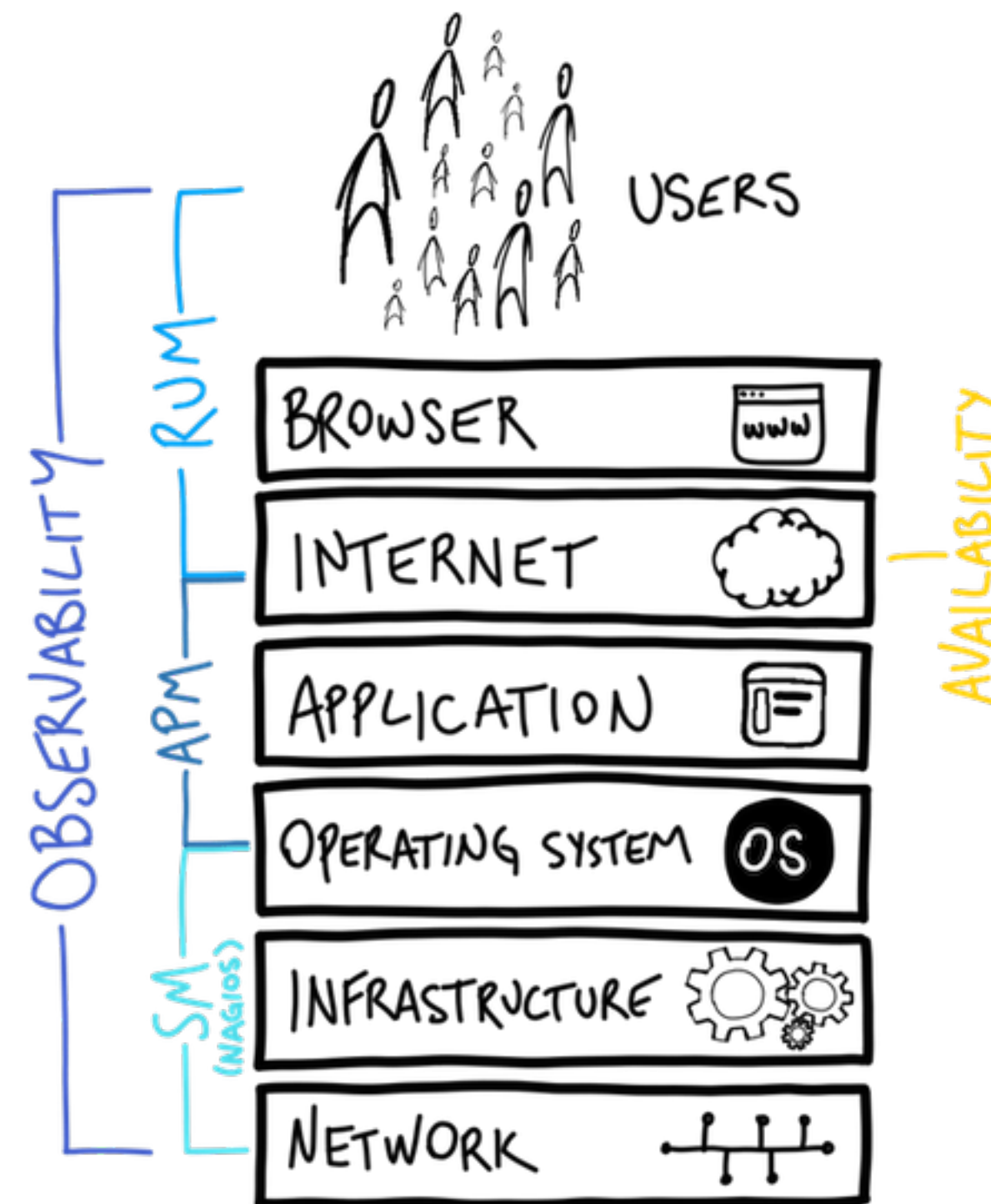
BACK OF THE QUEUE

REAL USER MONITORING

STILL ON THE RUM?



FRONT TO BACK (OPEN) TELEMETRY



<https://www.stack.io/blog/a-history-of-monitoring-and-opentelemetry>

JavaScript | OpenTelemetry

https://opentelemetry.io/docs/languages/js/

New Chrome available



DocsEcosystemStatusCommunityBlogEnglishSearch this site...

Docs

What is OpenTelemetry?

▶ Getting Started

▶ Concepts

▶ Demo

▼ Language APIs & SDKs

▶ SDK Config

▶ C++

▶ .NET

▶ Erlang/Elixir

▶ Go

▶ Java

▼ JavaScript

▶ Getting Started

Instrumentation

Libraries

Exporters

Context

Propagation

Resources

Sampling

Serverless

[Docs](#) / [Language APIs & SDKs](#) / JavaScript

JavaScript

 A language-specific implementation of OpenTelemetry in JavaScript (for Node.js & the browser).

This is the OpenTelemetry JavaScript documentation. OpenTelemetry is an observability framework – an API, SDK, and tools that are designed to aid in the generation and collection of application telemetry data such as metrics, logs, and traces. This documentation is designed to help you understand how to get started using OpenTelemetry JavaScript.

Status and Releases

The current status of the major functional components for OpenTelemetry JavaScript is as follows:

Traces

Metrics

Logs

[Stable](#) [Stable](#) [Development](#)

For releases, including the [latest release](#), see [Releases](#).

Warning

Client instrumentation for the browser is **experimental** and mostly **unspecified**. If you are interested in helping out, get in touch with the [Client Instrumentation SIG](#).



[View page source](#)

[Edit this page](#)

[Create documentation issue](#)

Status and Releases

Version Support

Repositories

Help or Feedback

Backlog · Client Instrumentation

github.com/orgs/open-telemetry/projects/19/views/1

Work

open-telemetry / Projects / Client Instrumentation

Type / to search

Off track

Client Instrumentation

Backlog

Filter by keyword or by field

Discard

No Status 1

semantic-conventions #1903

Clarify OS terminology and OS identifiers in device.app.lifecycle event

Icebox 0

Backlog 5

opentelemetry-js-contrib #2151

Implement browser Page Navigation Timing event instrumentation

opentelemetry-js-contrib #2140

Implement UserAction event instrumentation

opentelemetry-specification #3811

Standardize Server-Timing: traceparent "propagator" across vendors

opentelemetry-js-contrib #2152

Implement ResourceTiming event instrumentation for browser

opentelemetry-js-contrib #1461

Core Web Vitals Plugin

In Progress 3

opentelemetry-js-contrib #2372

Implement PageView event instrumentation

opentelemetry-js-contrib #2358

Session handling implementation

opentelemetry-js-contrib #2150

Implement event-based instrumentation for web errors

localhost:4173

localhost:4173

Finish update

 **OTel Records**

Records Events News

Featured



Loveless
My Bloody Valentine
Available formats



Licensed to Ill
Beastie Boys
Available formats



Hejira
Joni Mitchell
Available formats



Goo
Sonic Youth
Available formats



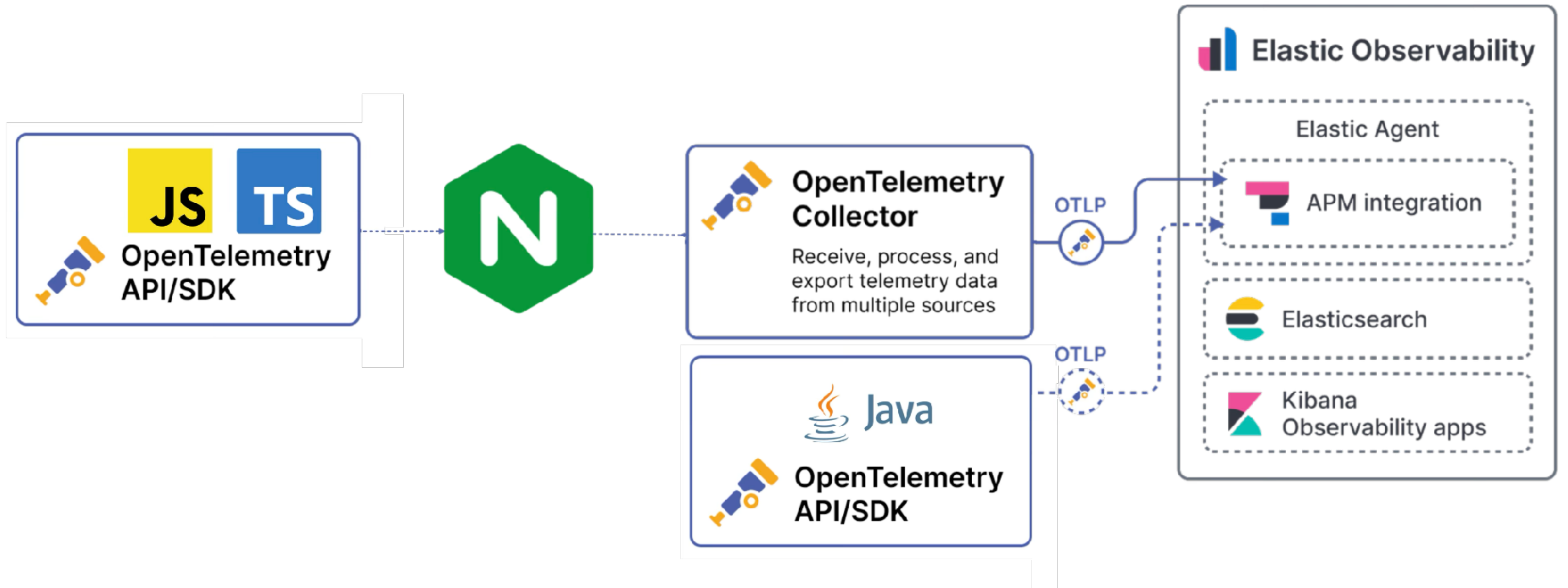
The Rise and Fall..
David Bowie
Available formats

Made by Carly Richmond with 💜 and excessive amounts of 🍷

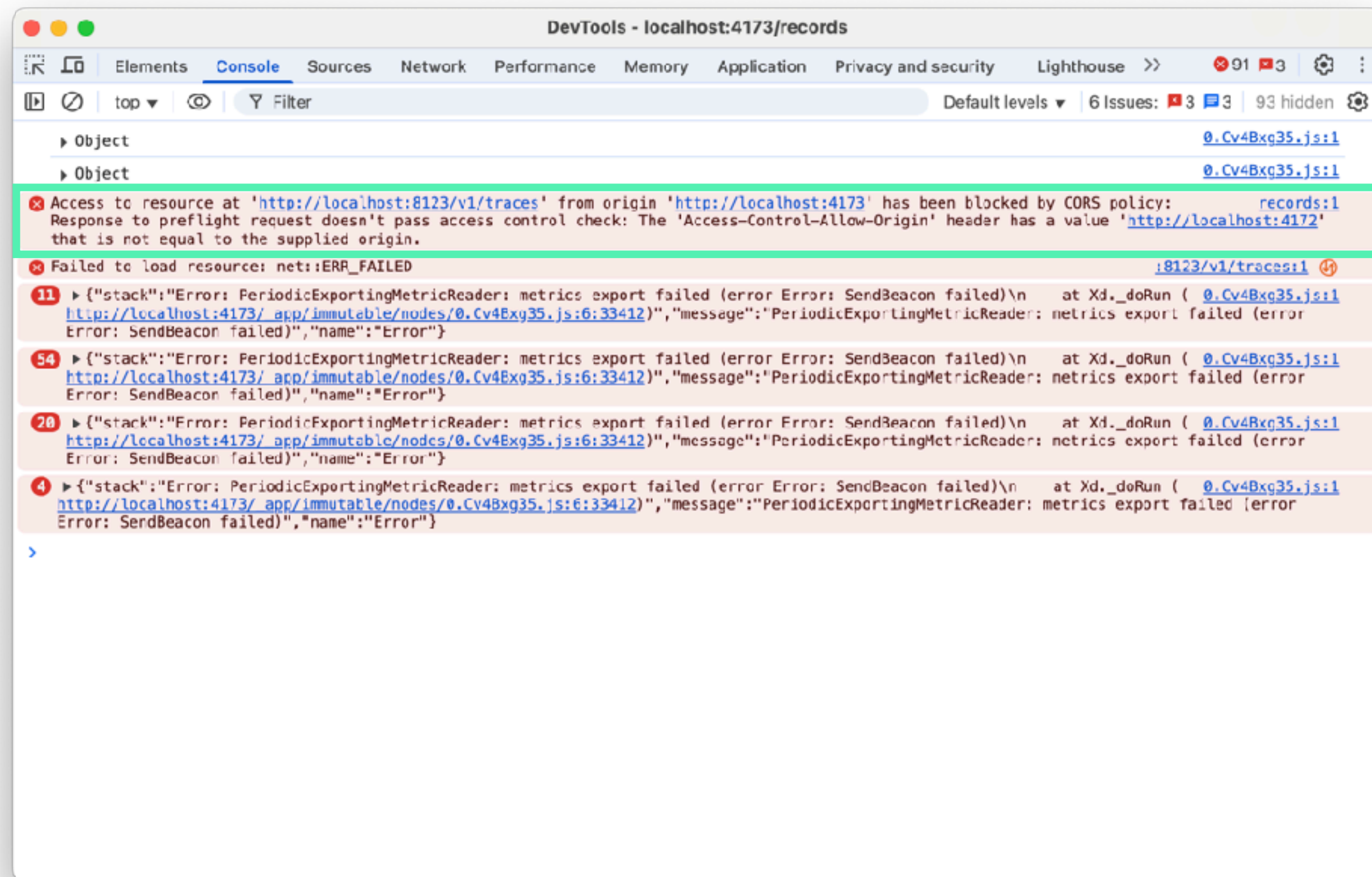


SCAN ME

OTEL WEB ARCHITECTURE



CUE CORS HELL!



NGINX PROXY

```
nginx.conf

1 events {}
2
3 http {
4
5     server {
6
7         listen 8123;
8
9         location /v1/traces {
10             proxy_pass http://host.docker.internal:4318;
11             # Apply CORS headers to ALL responses, including POST
12             add_header 'Access-Control-Allow-Origin' 'http://localhost:4173' always;
13             add_header 'Access-Control-Allow-Methods' 'POST, OPTIONS' always;
14             add_header 'Access-Control-Allow-Headers' 'Content-Type' always;
15             add_header 'Access-Control-Allow-Credentials' 'true' always;
16
17             # Preflight requests receive a 204 No Content response
18             if ($request_method = OPTIONS) {
19                 return 204;
20             }
21         }
22     }
23 }
```




```
1 /* OpenTelemetry Frontend packages */
2
3 // See https://github.com/carlyrichmond/otel-record-store for import list
4
5 // Define resource metadata for the service, used by exporters (Elastic requires service.name)
6 const SERVICE_NAME = 'records-ui-web';
7
8 const detectedResources = detectResources({ detectors: [browserDetector] });
9 let resource = resourceFromAttributes({
10     [ATTR_SERVICE_NAME]: SERVICE_NAME,
11     'service.version': 1,
12     'deployment.environment': 'dev'
13 });
14 resource = resource.merge(detectedResources);
15
16 // Configure logging to send to the collector via nginx
17 const logExporter = new OTLPLogExporter({
18     url: LOGS_URL // nginx proxy
19 });
20
21 const loggerProvider = new LoggerProvider({
22     resource: resource,
23     processors: [new BatchLogRecordProcessor(logExporter)]
24 });
25
26 logs.setGlobalLoggerProvider(loggerProvider);
27
```


Bayfield St

Logs

Metrics

Traces

Bayfield St

Logs

Metrics

Traces

ECS Logging with Winston | E x +

← → ↺ https://www.elastic.co/guide/en/ecs-logging/nodejs/current/winston.html

elastic Platform Solutions Customers Resources Pricing Docs

🔍 👤 Start free trial Contact Sales

Introduction

ECS Logging with Pino

ECS Logging with Winston

ECS Logging with Morgan

Elastic Docs › ECS Logging: Node.js Reference ▾

ECS Logging with Winston

This Node.js package provides a formatter for the [winston](#) logger, compatible with [Elastic Common Schema \(ECS\) logging](#). In combination with the [Filebeat](#) shipper, you can [monitor all your logs](#) in one place in the Elastic Stack. `winston` 3.x versions `>=3.3.3` are supported.

Setup

Step 1: Install

```
$ npm install @elastic/ecs-winston-format
```

Step 2: Configure

```
const winston = require('winston');
const { ecsFormat } = require('@elastic/ecs-winston-format');

const loader = winston.createLogger({
```

edit

edit

edit

On this page

Setup

Step 1: Install

Step 2: Configure

Step 3: Configure Filebeat

Usage

Error logging

HTTP Request and Response Logging

Log Correlation with APM

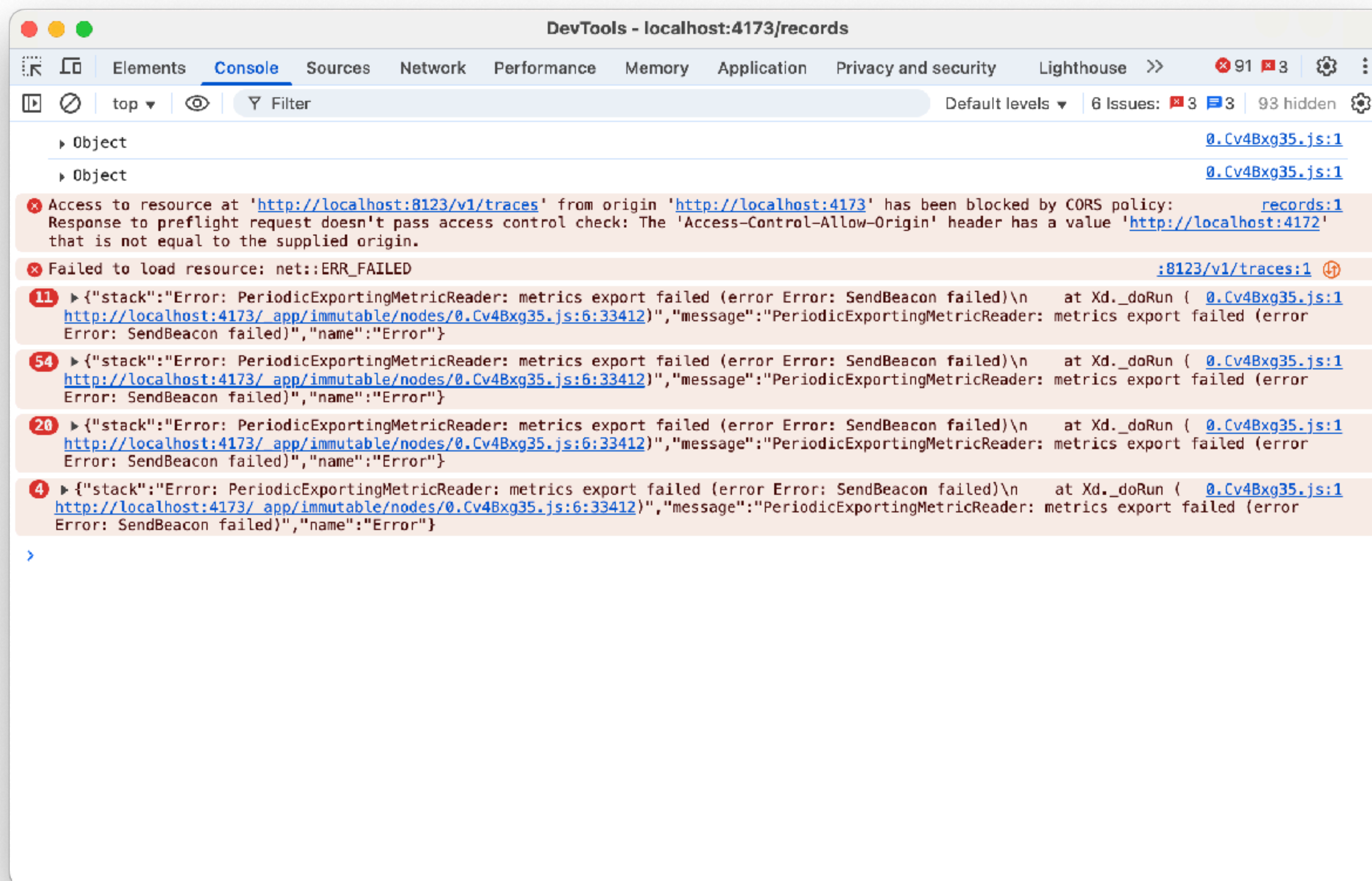
Limitations and Considerations

ElasticON events are back!

Learn from Elastic experts and discover how to supercharge app development, security analytics, log analytics, and more — with the Elastic Search AI Platform.

Learn more

DEVTOOLS CONSOLE ISN'T VISIBLE!





```
1 /* OpenTelemetry Frontend packages */
2
3 // See https://github.com/carlyrichmond/otel-record-store for import list
4
5 // Define resource metadata for the service, used by exporters (Elastic requires service.name)
6 const SERVICE_NAME = 'records-ui-web';
7
8 const detectedResources = detectResources({ detectors: [browserDetector] });
9 let resource = resourceFromAttributes({
10     [ATTR_SERVICE_NAME]: SERVICE_NAME,
11     'service.version': 1,
12     'deployment.environment': 'dev'
13 });
14 resource = resource.merge(detectedResources);
15
16 // Configure logging to send to the collector via nginx
17 const logExporter = new OTLPLogExporter({
18     url: LOGS_URL // nginx proxy
19 });
20
21 const loggerProvider = new LoggerProvider({
22     resource: resource,
23     processors: [new BatchLogRecordProcessor(logExporter)]
24 });
25
26 logs.setGlobalLoggerProvider(loggerProvider);
27
```



```
11     'service.version': 1,  
12     'deployment.environment': 'dev'  
13 });  
14 resource = resource.merge(detectedResources);  
15  
16 // Configure logging to send to the collector via nginx  
17 const logExporter = new OTLPLogExporter({  
18     url: LOGS_URL // nginx proxy  
19 });  
20  
21 const loggerProvider = new LoggerProvider({  
22     resource: resource,  
23     processors: [new BatchLogRecordProcessor(logExporter)]  
24 });  
25  
26 logs.setGlobalLoggerProvider(loggerProvider);  
27  
28 const logger = logs.getLogger('default', '1.0.0');  
29 logger.emit({  
30     severityNumber: SeverityNumber.INFO,  
31     severityText: 'INFO',  
32     body: 'Logger initialized'  
33 });  
34  
35 // Configure the OTLP exporter to talk to the collector via nginx  
36 const exporter = new OTLPTraceExporter({  
37     url: TRACE_URL // nginx proxy  
38 });  
39  
40 // Instantiate the trace provider and inject the resource
```


All logs

🔍 service.name : "records-ui-web"

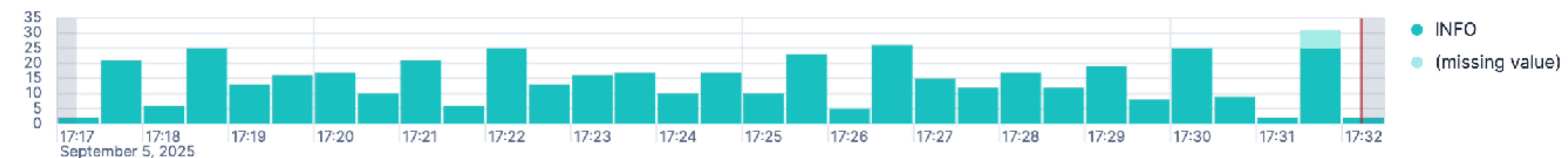
  Last 15 minutes

 Refresh

 Search field names

$$\frac{1}{2} \quad 0$$

 Auto interval

Breakdown by log.level 

Sep 5, 2025 @ 17:17:13.637 - Sep 5, 2025 @ 17:32:13.637 (interval: Auto - 30 seconds)

Field statistics

Columns 4

Sort fields 1



Actions

@timestamp

↓ **body.text**

k severity_text

severity_number

🔍 ↗️ 📄 📅 Sep 5, 2025 @ 17:32:03.305 Client Telemetry Initialised INFO 9




 Sep 5, 2025 @ 17:32:03.296 Logger initialized INFO 9

🔍 ↗ 📄 📅 Sep 5, 2025 @ 17:31:58.489 Client Telemetry Initialised INFO 9





 Sep 5, 2025 @ 17:31:58.480 Logger initialized INFO 9

🔍 ↗ 📄 📅 Sep 5, 2025 @ 17:31:57.736 Client Telemetry Initialised INFO 9





 Sep 5, 2025 @ 17:31:57.727 Logger initialized INFO 9

🔍 ↗ 📄 📅 Sep 5, 2025 @ 17:31:53.188 Client Telemetry Initialised INFO 9

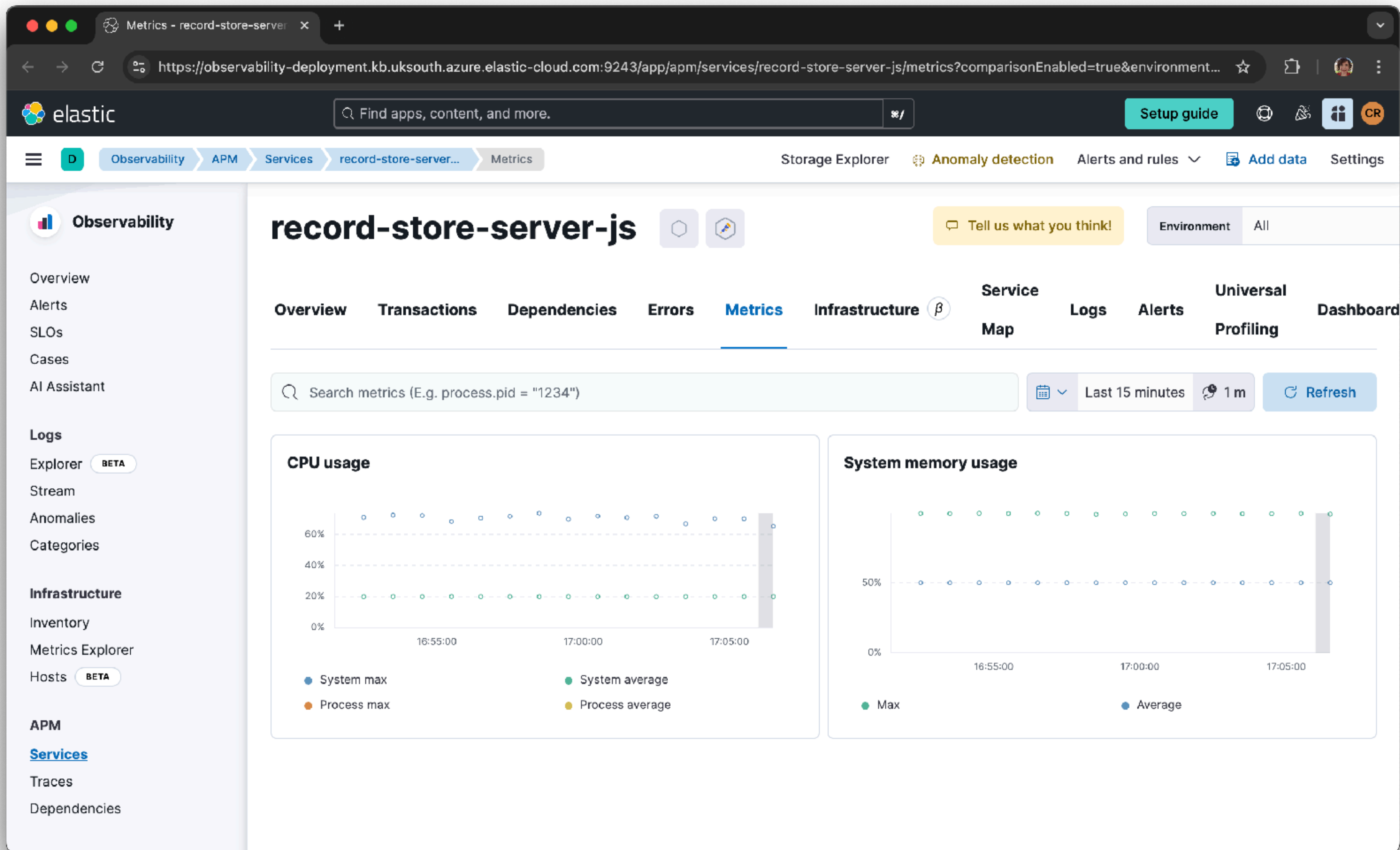
 Add a field

Bayfield St

Logs

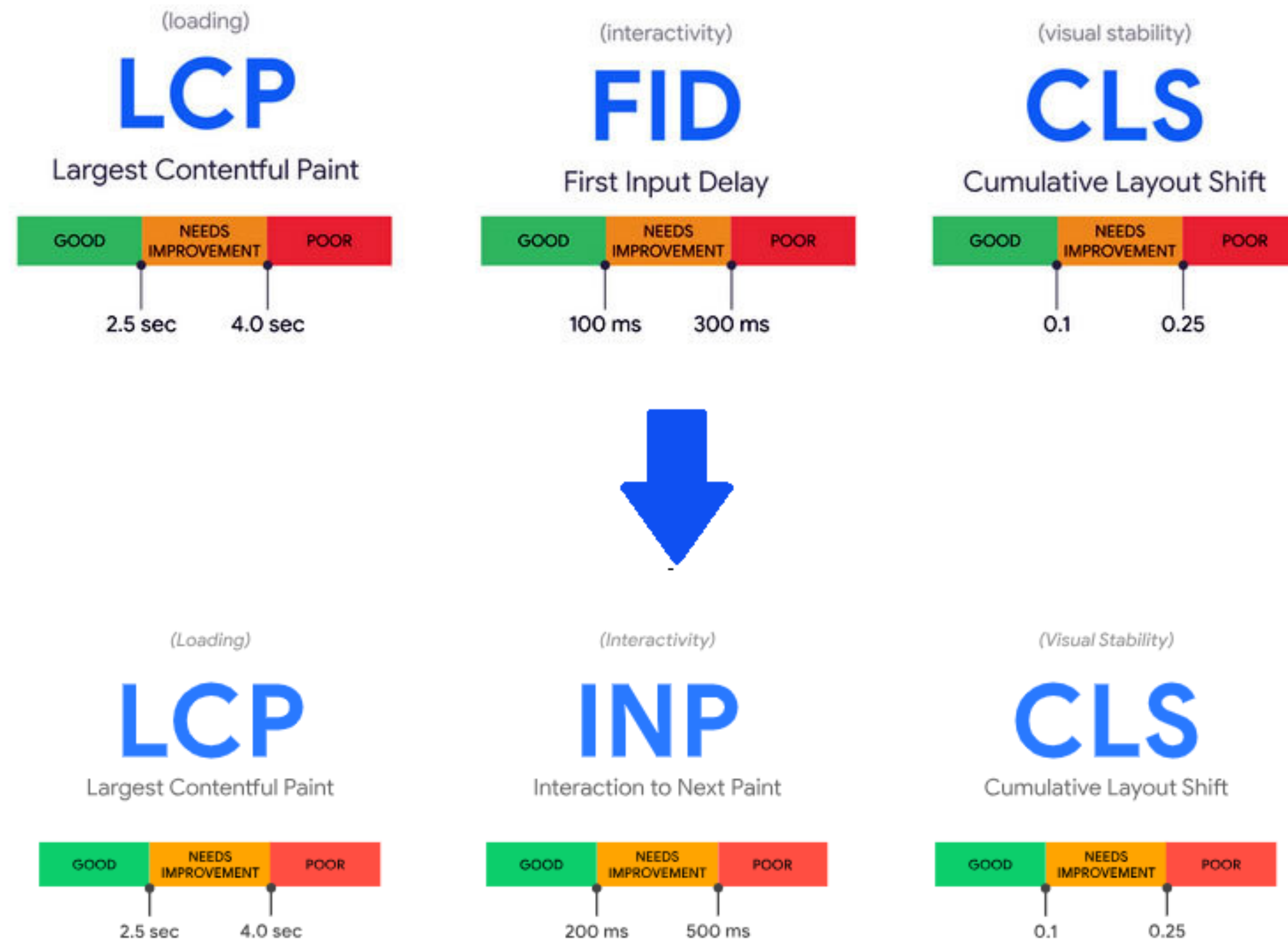
Metrics

Traces



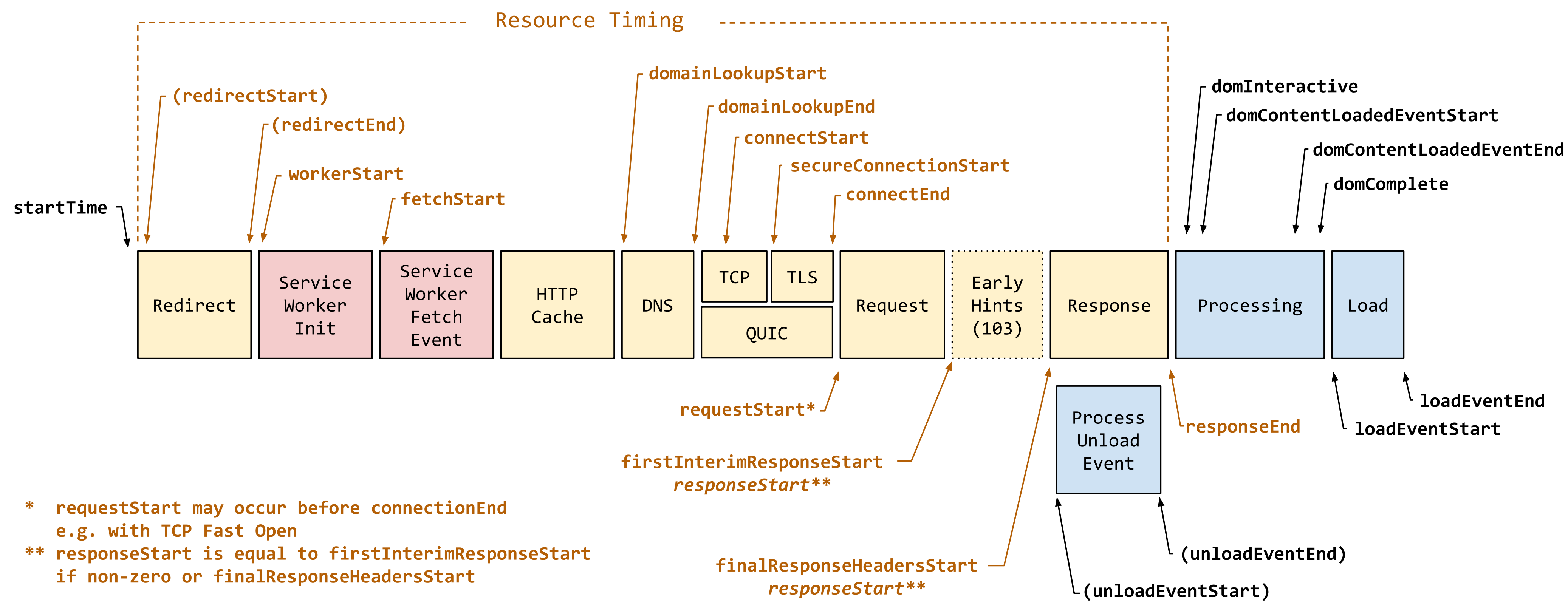


GOOGLE CORE WEB VITALS



OTHER VITALS

PAGE LOAD

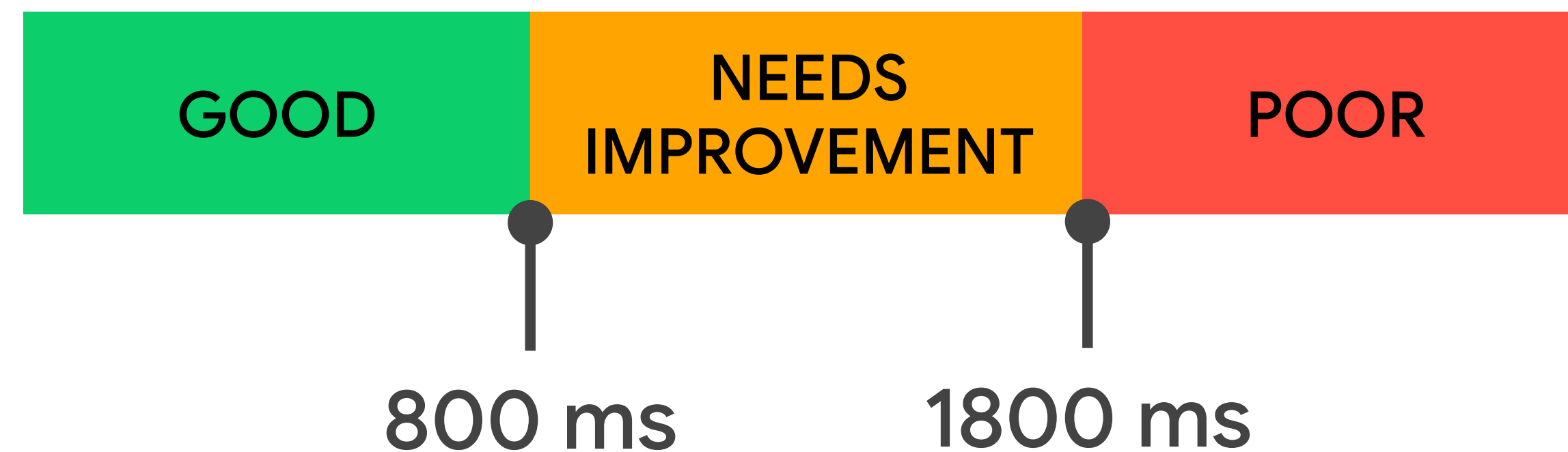


<https://web.dev/articles/ttfb>

TTFB

TIME TO FIRST BYTE

The time between the request for a resource being initiated and when the first byte of a response begins to arrive.



<https://web.dev/articles/ttfb>

web-vitals - npm

https://www.npmjs.com/package/web-vitals#overview

Pro

Teams

Pricing

Documentation

npm

Search packages

Search

Sign Up

Log In

web-vitals

TS

4.2.3 • Public • Published 2 months ago

Readme

Code

Beta

0 Dependencies

18,887 Dependents

76 Versions

web-vitals

Overview

Install and load the library

- From npm
- From a CDN

Usage

- Basic usage
- Report the value on every change
- Report only the delta of changes
- Send the results to an analytics endpoint
- Send the results to Google Analytics
- Send the results to Google Tag Manager
- Send attribution data
- Batch multiple reports together

Install

> npm i web-vitals

Repository

github.com/GoogleChrome/web-vitals

Homepage

github.com/GoogleChrome/web-vitals#...

Weekly Downloads

4,090,498

Version

4.2.3

License

Apache-2.0

Unpacked Size

Total Files

elastic

carlyrichmond.bsky.social


```
1 /* Web Vitals Frontend package*/
2
3 // See https://github.com/carlyrichmond/otel-record-store for import list
4
5 type CWVMetric = LCPMetric | CLSMetric | INPMetric | TTFBMetric | FCPMetric;
6
7 export class WebVitalsInstrumentation extends InstrumentationBase {
8
9     private cwvMeter: Meter;
10
11     /* Core Web Vitals Measures */
12     private lcp: ObservableGauge;
13     private cls: ObservableGauge;
14     private inp: ObservableGauge;
15     private ttfb: ObservableGauge;
16     private fcp: ObservableGauge;
17
18     constructor(config: InstrumentationConfig, resource: Resource) {
19         super('WebVitalsInstrumentation', '1.0', config);
20
21         const myServiceMeterProvider = this.generateMeterForResource(resource);
22         metrics.setGlobalMeterProvider(myServiceMeterProvider);
23
24         this.cwvMeter = metrics.getMeter('core-web-vitals', '1.0.0');
25
26         // Initialising CWV metric instruments
27         this.lcp = this.cwvMeter.createObservableGauge('lcp', { unit: 'ms', description: 'Largest Contentful Paint' });
28         this.cls = this.cwvMeter.createObservableGauge('cls', { description: 'Cumulative Layout Shift' });
29         this.inp = this.cwvMeter.createObservableGauge('inp', { unit: 'ms', description: 'Interaction to Next Paint' });
30         this.ttfb = this.cwvMeter.createObservableGauge('ttfb', { unit: 'ms', description: 'Time to First Byte' });
```



```

18 constructor(config: InstrumentationConfig, resource: Resource) {
19     super('WebVitalsInstrumentation', '1.0', config);
20
21     const myServiceMeterProvider = this.generateMeterForResource(resource);
22     metrics.setGlobalMeterProvider(myServiceMeterProvider);
23
24     this.cwvMeter = metrics.getMeter('core-web-vitals', '1.0.0');
25
26     // Initialising CWV metric instruments
27     this.lcp = this.cwvMeter.createObservableGauge('lcp', { unit: 'ms', description: 'Largest Contentful Paint' });
28     this.cls = this.cwvMeter.createObservableGauge('cls', { description: 'Cumulative Layout Shift' });
29     this.inp = this.cwvMeter.createObservableGauge('inp', { unit: 'ms', description: 'Interaction to Next Paint' });
30     this.ttfb = this.cwvMeter.createObservableGauge('ttfb', { unit: 'ms', description: 'Time to First Byte' });
31     this.fcp = this.cwvMeter.createObservableGauge('fcp', { unit: 'ms', description: 'First Contentful Paint' });
32 }
33
34 protected init(): InstrumentationModuleDefinition | InstrumentationModuleDefinition[] | void {}
35
36 // Creating Meter Provider factory to send metrics via OTLP
37 private generateMeterForResource(resource: Resource) {
38     const metricReader = new PeriodicExportingMetricReader({
39         exporter: new OTLPMetricExporter({
40             url: METRICS_URL //nginx proxy
41         }),
42         // Default is 60000ms (60 seconds).
43         // Set to 10 seconds for demo purposes only.
44         exportIntervalMillis: 10000
45     });
46
47     return new MeterProvider({
48         resource: resource,
49         readers: [metricReader]
50     });

```



```
53 enable() {
54     // Capture Largest Contentful Paint
55     onLCP(
56         (metric) => {
57             this.lcp.addCallback((result) => {
58                 this.sendMetric(metric, result);
59             });
60         },
61         { reportAllChanges: true }
62     );
63
64     // Capture Cumulative Layout Shift
65     onCLS(
66         (metric) => {
67             this.cls.addCallback((result) => {
68                 this.sendMetric(metric, result);
69             });
70         },
71         { reportAllChanges: true }
72     );
73
74     // Capture Interaction to Next Paint
75     onINP(
76         (metric) => {
77             this.inp.addCallback((result) => {
78                 this.sendMetric(metric, result);
79             });
80         },
81         { reportAllChanges: true }
82     );
83
84     // Time to First Byte
85     onTTFB(
```



```

90     },
91     { reportAllChanges: true }
92 );
93
94 // First Contentful Paint
95 onFCP(
96     (metric) => {
97         this.fcp.addCallback((result) => {
98             this.sendMetric(metric, result);
99         });
100     },
101     { reportAllChanges: true }
102 );
103 }
104

```

```

105 private sendMetric(metric: CWVMetric, result: ObservableResult<Attributes>): void {
106     const now = hrTime();
107
108     const attributes = {
109         startTime: now,
110         'web_vital.name': metric.name,
111         'web_vital.id': metric.id,
112         'web_vital.navigationType': metric.navigationType,
113         'web_vital.delta': metric.delta,
114         'web_vital.value': metric.value,
115         'web_vital.rating': metric.rating,
116         // metric specific attributes
117         'web_vital.entries': JSON.stringify(metric.entries)
118     };
119
120     result.observe(metric.value, attributes);
121 }
122 }

```

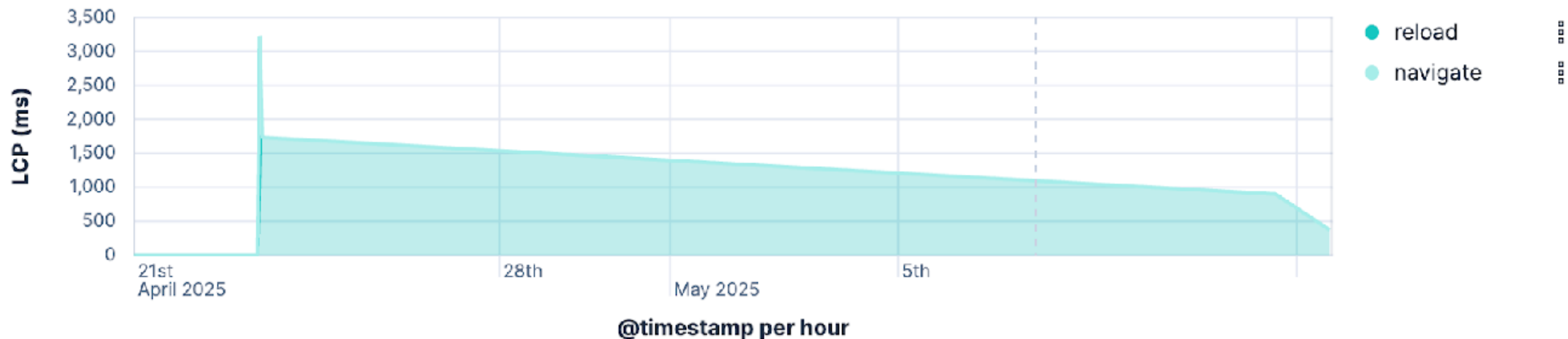

OTel Core Web Vitals Summary

This dashboard shows the results of Core Web Vitals captured from the OTEL record store application using the [web-vitals JS library](#) and [OpenTelemetry JavaScript SDK](#).

Full instrumentation code available [here](#).

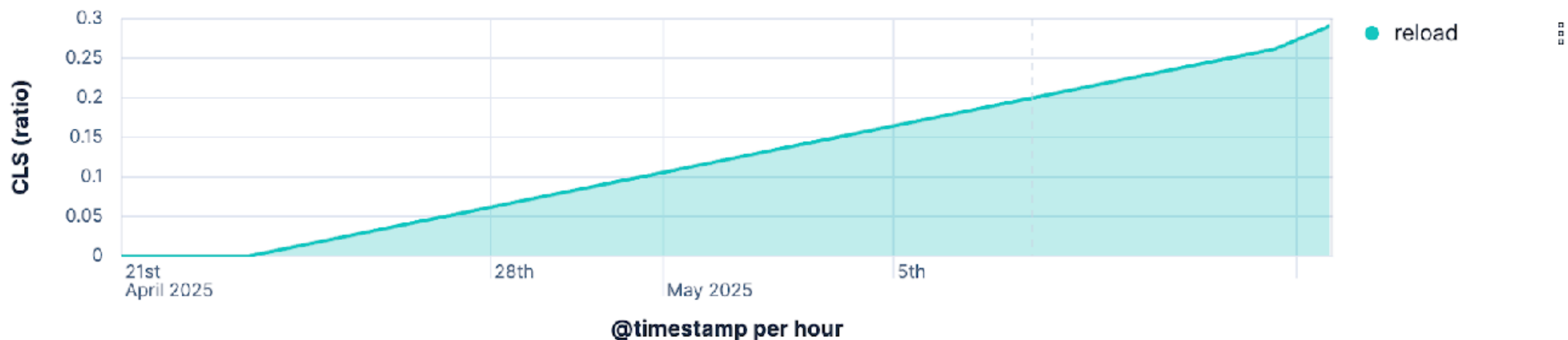
Largest Contentful Paint (LCP)

good
372



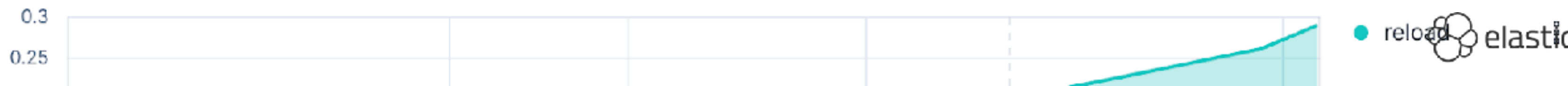
Cumulative Layout Shift (CLS)

poor
0.291



Interaction to Next Paint (INP)

Exit full screen



Bayfield St

Logs

Metrics

Traces

Latency distribution ⓘ | 38 total transactions

ⓘ Click and drag to select a range



Trace sample

⏪ < 1 of 38 > ⏩

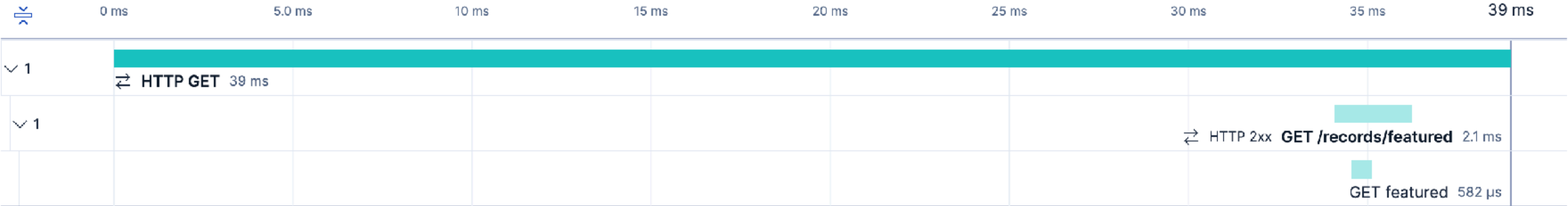
Investigate ▾ View full trace

17 seconds ago | 39 ms (100% of trace)

Timeline Metadata Logs

☒ Show critical path

Services records-ui-web record-store-server-java

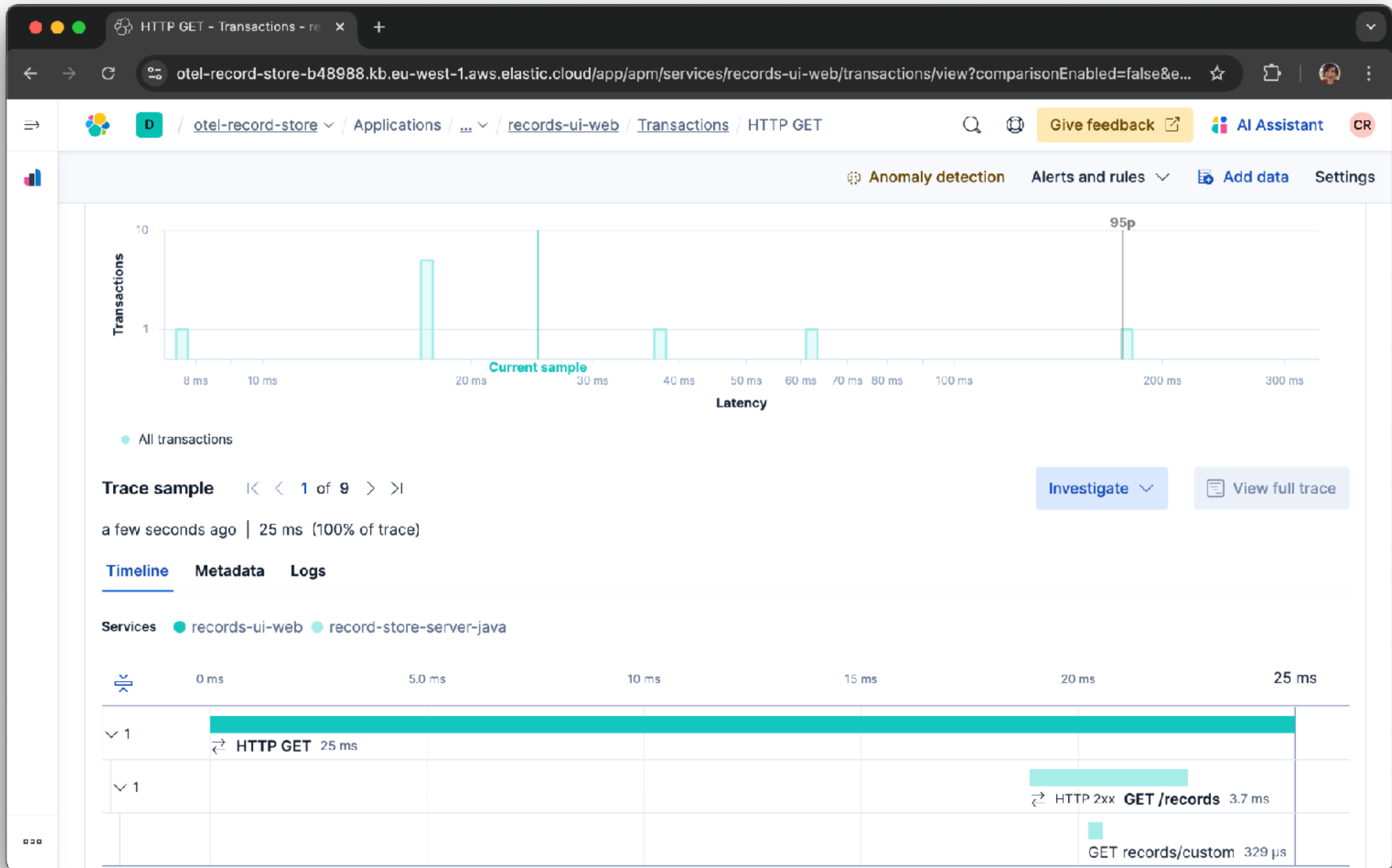



```
28 const logger = logs.getLogger('default', '1.0.0');
29 logger.emit({
30     severityNumber: SeverityNumber.INFO,
31     severityText: 'INFO',
32     body: 'Logger initialized'
33 });
34
35 // Configure the OTLP exporter to talk to the collector via nginx
36 const exporter = new OTLPTraceExporter({
37     url: TRACE_URL // nginx proxy
38 });
39
40 // Instantiate the trace provider and inject the resource
41 const provider = new WebTracerProvider({
42     resource: resource,
43     spanProcessors: [
44         // Send each completed span through the exporter
45         new BatchSpanProcessor(exporter),
46         new BatchSpanProcessor(new ConsoleSpanExporter())
47     ]
48 });
49
50 // Register the provider with propagation and set up the async context manager for spans
51 provider.register({
52     contextManager: new ZoneContextManager(),
53     propagator: new CompositePropagator({
54         propagators: [new W3CBaggagePropagator(), new W3CTraceContextPropagator()]
55     })
56 });
57
```



```
39
40 // Instantiate the trace provider and inject the resource
41 const provider = new WebTracerProvider({
42     resource: resource,
43     spanProcessors: [
44         // Send each completed span through the exporter
45         new BatchSpanProcessor(exports),
46         new BatchSpanProcessor(new ConsoleSpanExporter())
47     ]
48 });
49
50 // Register the provider with propagation and set up the async context manager for spans
51 provider.register({
52     contextManager: new ZoneContextManager(),
53     propagator: new CompositePropagator({
54         propagators: [new W3CBaggagePropagator(), new W3CTraceContextPropagator()]
55     })
56 });
57
58 export class ClientTelemetry {
59     private static instance: ClientTelemetry;
60     private initialized = false;
61
62     private constructor() {}
63
64     // Obtain singleton instance of provider
65     public static getInstance(): ClientTelemetry {
66         if (!ClientTelemetry.instance) {
67             ClientTelemetry.instance = new ClientTelemetry();
68             ClientTelemetry.instance.start();
```


WE ALL HAVE BAGGAGE!





```
1 /* OpenTelemetry Frontend packages */
2
3 // See https://github.com/carlyrichmond/otel-record-store for import list
4
5 // Define resource metadata for the service, used by exporters (Elastic requires service.name)
6 const SERVICE_NAME = 'records-ui-web';
7
8 const detectedResources = detectResources({ detectors: [browserDetector] });
9 let resource = resourceFromAttributes({
10     [ATTR_SERVICE_NAME]: SERVICE_NAME,
11     'service.version': 1,
12     'deployment.environment': 'dev'
13 });
14 resource = resource.merge(detectedResources);
15
16 // Configure logging to send to the collector via nginx
17 const logExporter = new OTLPLogExporter({
18     url: LOGS_URL // nginx proxy
19 });
20
21 const loggerProvider = new LoggerProvider({
22     resource: resource,
23     processors: [new BatchLogRecordProcessor(logExporter)]
24 });
25
26 logs.setGlobalLoggerProvider(loggerProvider);
27
```



```
72
73 // Initializer
74 public start() {
75     if (!this.initialized) {
76         // Enable automatic span generation for document load and user click interactions
77         registerInstrumentations({
78             instrumentations: [
79                 // Automatically tracks when the document loads
80                 new DocumentLoadInstrumentation({
81                     ignoreNetworkEvents: false,
82                     ignorePerformancePaintEvents: false
83                 }),
84                 getWebAutoInstrumentations({
85                     '@opentelemetry/instrumentation-fetch': {
86                         propagateTraceHeaderCorsUrls: /.*/,
87                         clearTimingResources: true
88                     },
89                     '@opentelemetry/instrumentation-xml-http-request': {
90                         propagateTraceHeaderCorsUrls: /.*/
91                     }
92                 }),
93                 // User events
94                 new UserInteractionInstrumentation({
95                     eventNames: ['click', 'input'] // instrument click and input events only
96                 }),
97                 // Custom Web Vitals instrumentation
98                 new WebVitalsInstrumentation({}, resource)
99             ]
100         });
101     }
```



```
72
73 // Initializer
74 public start() {
75     if (!this.initialized) {
76         // Enable automatic span generation for document load and user click interactions
77         registerInstrumentations({
78             instrumentations: [
79                 // Automatically tracks when the document loads
80                 new DocumentLoadInstrumentation({
81                     ignoreNetworkEvents: false,
82                     ignorePerformancePaintEvents: false
83                 }),
84                 getWebAutoInstrumentations({
85                     '@opentelemetry/instrumentation-fetch': {
86                         propagateTraceHeaderCorsUrls: /* */,
87                         clearTimingResources: true
88                     },
89                     '@opentelemetry/instrumentation-xml-http-request': {
90                         propagateTraceHeaderCorsUrls: /* */
91                     }
92                 }),
93                 // User events
94                 new UserInteractionInstrumentation({
95                     eventNames: ['click', 'input'] // instrument click and input events only
96                 }),
97                 // Custom Web Vitals instrumentation
98                 new WebVitalsInstrumentation({}, resource)
99             ]
100         });
101     }
```


Latency distribution ⓘ | 38 total transactions

ⓘ Click and drag to select a range



Trace sample |< < 1 of 38 > >|

Investigate

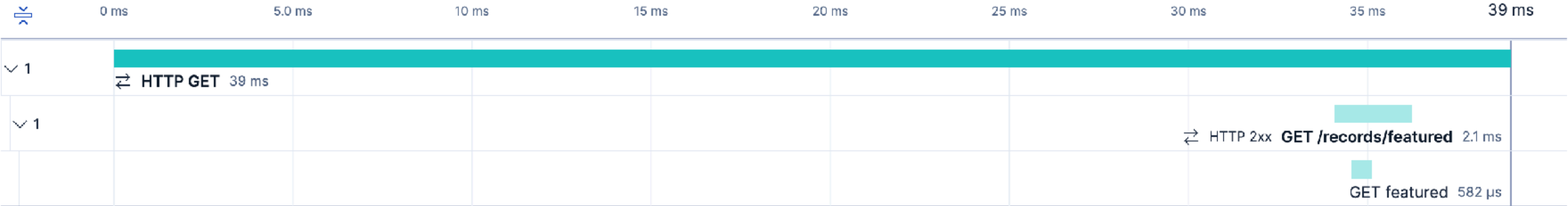
View full trace

17 seconds ago | 39 ms (100% of trace)

Timeline Metadata Logs

☒ Show critical path

Services records-ui-web record-store-server-java




```
72
73 // Initializer
74 public start() {
75     if (!this.initialized) {
76         // Enable automatic span generation for document load and user click interactions
77         registerInstrumentations({
78             instrumentations: [
79                 // Automatically tracks when the document loads
80                 new DocumentLoadInstrumentation({
81                     ignoreNetworkEvents: false,
82                     ignorePerformancePaintEvents: false
83                 }),
84                 getWebAutoInstrumentations({
85                     '@opentelemetry/instrumentation-fetch': {
86                         propagateTraceHeaderCorsUrls: /* */,
87                         clearTimingResources: true
88                     },
89                     '@opentelemetry/instrumentation-xml-http-request': {
90                         propagateTraceHeaderCorsUrls: /* */
91                     }
92                 }),
93                 // User events
94                 new UserInteractionInstrumentation({
95                     eventNames: ['click', 'input'] // instrument click and input events only
96                 }),
97                 // Custom Web Vitals instrumentation
98                 new WebVitalsInstrumentation({}, resource)
99             ]
100         });
101     }
```


- All transactions

Investigate ▾

 [View full trace](#)

Timeline Metadata Logs

Type ● records-ui-web ● http



All transactions

Trace sample

1 of 9

a month ago | 230 ms (100% of trace)

Timeline

Metadata

Logs

Show critical path

Type

records-ui-web

http

0 s

50 s

7

Success documentLoad 230 ms

Span details

View span in Discover

Name

resourceFetch

Dependency

localhost:4173

Service

records-ui-web

Transaction

documentLoad

a month ago | 11 ms (0.0% of transaction) | external http

Metadata

Filter...

How to add labels and other data

@timestamp

@timestamp

2025-07-25T12:09:58.941Z

agent

agent.name

otlp

agent.version

unknown

attributes

attributes.event.outcome

success

attributes.event.success_count

1

attributes.http.response_content_length

13262

attributes.http.url

http://localhost:4173/src/app.css?t=1753445398940

attributes.processor.event

span

attributes.service.target.name

localhost:4173

attributes.service.target.name.text

localhost:4173

attributes.service.target.type

http


```
72
73 // Initializer
74 public start() {
75     if (!this.initialized) {
76         // Enable automatic span generation for document load and user click interactions
77         registerInstrumentations({
78             instrumentations: [
79                 // Automatically tracks when the document loads
80                 new DocumentLoadInstrumentation({
81                     ignoreNetworkEvents: false,
82                     ignorePerformancePaintEvents: false
83                 }),
84                 getWebAutoInstrumentations({
85                     '@opentelemetry/instrumentation-fetch': {
86                         propagateTraceHeaderCorsUrls: /* */,
87                         clearTimingResources: true
88                     },
89                     '@opentelemetry/instrumentation-xml-http-request': {
90                         propagateTraceHeaderCorsUrls: /* */
91                     }
92                 }),
93                 // User events
94                 new UserInteractionInstrumentation({
95                     eventNames: ['click', 'input'] // instrument click and input events only
96                 }),
97                 // Custom Web Vitals instrumentation
98                 new WebVitalsInstrumentation({}, resource)
99             ]
100         });
101     }
```

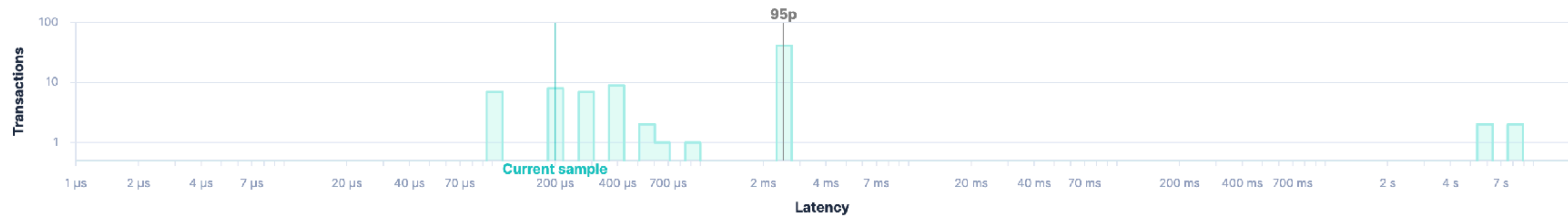

Trace samples

Latency correlations

Failed transaction correlations

Latency distribution ⓘ | 85 total transactions

ⓘ Click and drag to select a range



All transactions

Trace sample

1 of 85

Investigate

View full trace

19 minutes ago | 200 μs (100% of trace)

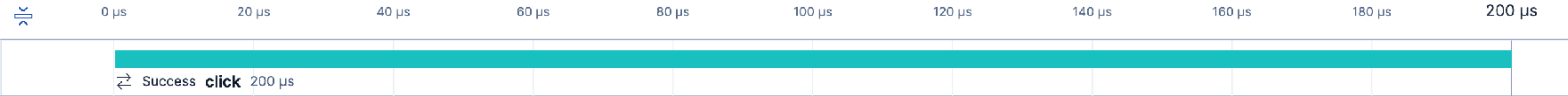
Timeline

Metadata

Logs

Show critical path

Type records-ui-web



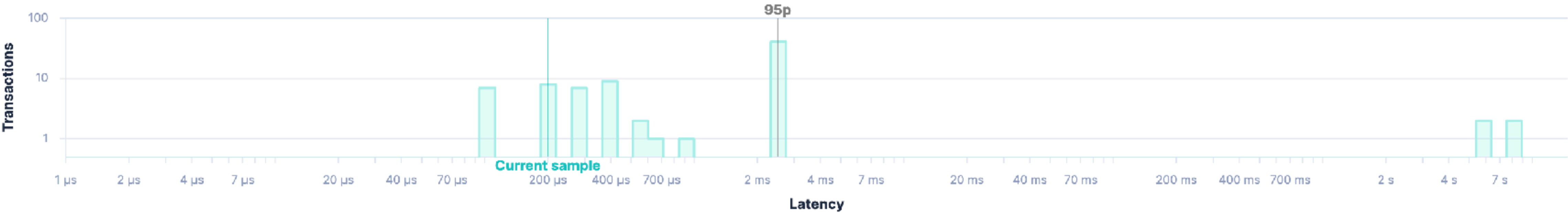
Trace samples

Latency correlations

Failed transaction correlations

Latency distribution ⓘ 85 total transactions

ⓘ Click and drag to select a range



All transactions

Trace sample

⏪

<

1 of 85

>

⏩

Investigate

View full trace

20 minutes ago | 200 μs (100% of trace)

Timeline

Metadata

Logs

🔍 xpath

×

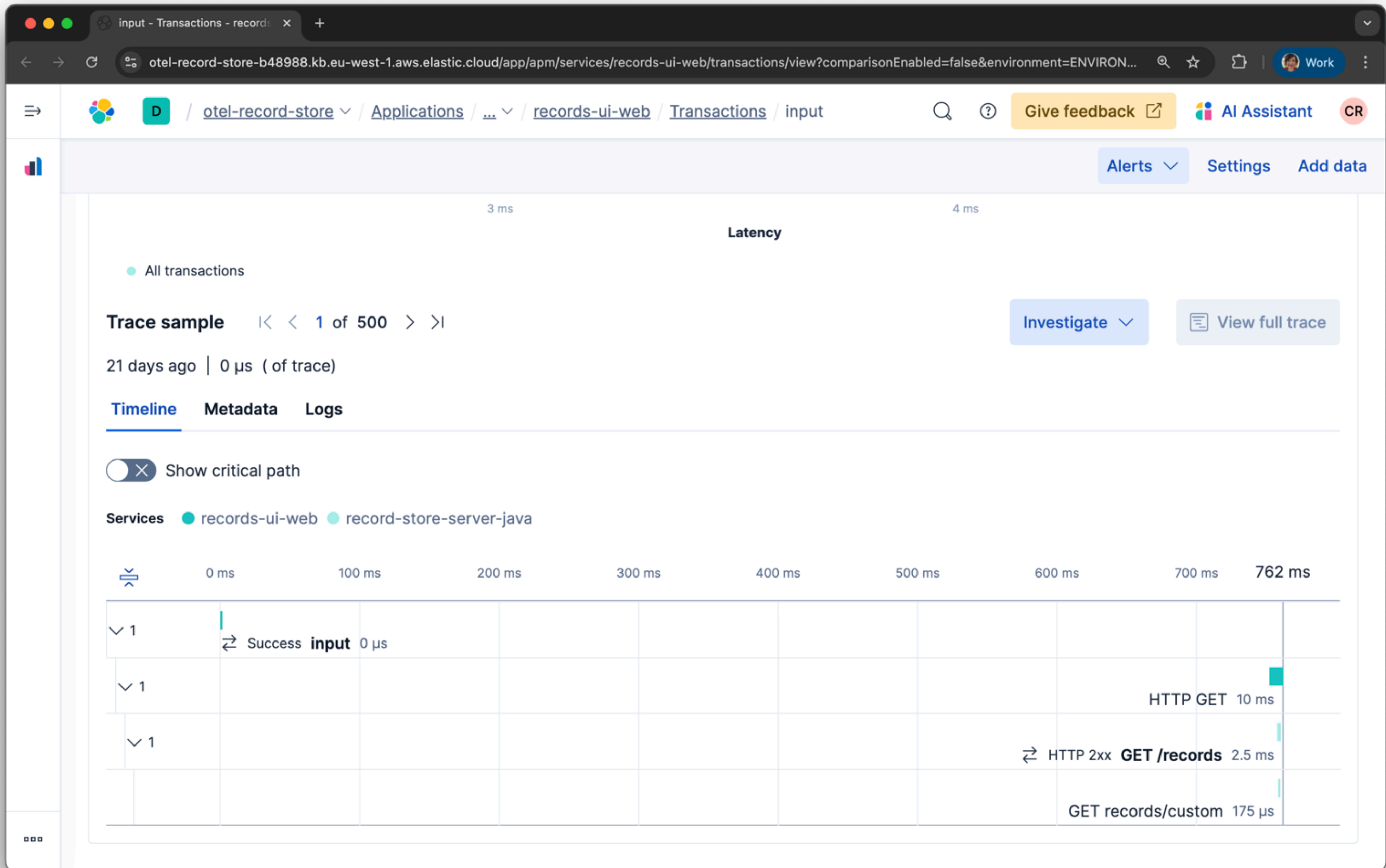
ⓘ How to add labels and other data

attributes

attributes.target_xpath	//html/body/div/section/div/button
-------------------------	------------------------------------

target_xpath

target_xpath	//html/body/div/section/div/button
--------------	------------------------------------



JavaScript | OpenTelemetry

https://opentelemetry.io/docs/languages/js/

New Chrome available



DocsEcosystemStatusCommunityBlogEnglishSearch this site...

Docs

What is OpenTelemetry?

▶ Getting Started

▶ Concepts

▶ Demo

▼ Language APIs & SDKs

▶ SDK Config

▶ C++

▶ .NET

▶ Erlang/Elixir

▶ Go

▶ Java

▼ JavaScript

▶ Getting Started

Instrumentation

Libraries

Exporters

Context

Propagation

Resources

Sampling

Serverless

[Docs](#) / [Language APIs & SDKs](#) / JavaScript

JavaScript

 A language-specific implementation of OpenTelemetry in JavaScript (for Node.js & the browser).

This is the OpenTelemetry JavaScript documentation. OpenTelemetry is an observability framework – an API, SDK, and tools that are designed to aid in the generation and collection of application telemetry data such as metrics, logs, and traces. This documentation is designed to help you understand how to get started using OpenTelemetry JavaScript.

Status and Releases

The current status of the major functional components for OpenTelemetry JavaScript is as follows:

Traces

Metrics

Logs

[Stable](#) [Stable](#) [Development](#)

For releases, including the [latest release](#), see [Releases](#).

Warning

Client instrumentation for the browser is **experimental** and mostly **unspecified**. If you are interested in helping out, get in touch with the [Client Instrumentation SIG](#).



[View page source](#)

[Edit this page](#)

[Create documentation issue](#)

Status and Releases

Version Support

Repositories

Help or Feedback

Bayfield St

Logs

Metrics

Traces

Thank you!



SCAN ME